

Arquitectura MVC - Resumen Ejecutivo

Implementación Completada

Lo que se ha creado:

...

CONTROLADORES (5)

- └─ DocumentosController  200+ líneas
- └─ DocumentoDetailController  70 líneas
- └─ CarpetasController  60 líneas
- └─ UsuariosController  80 líneas
- └─ PermisosController  150 líneas

VISTAS (1 completa + 4 widgets)

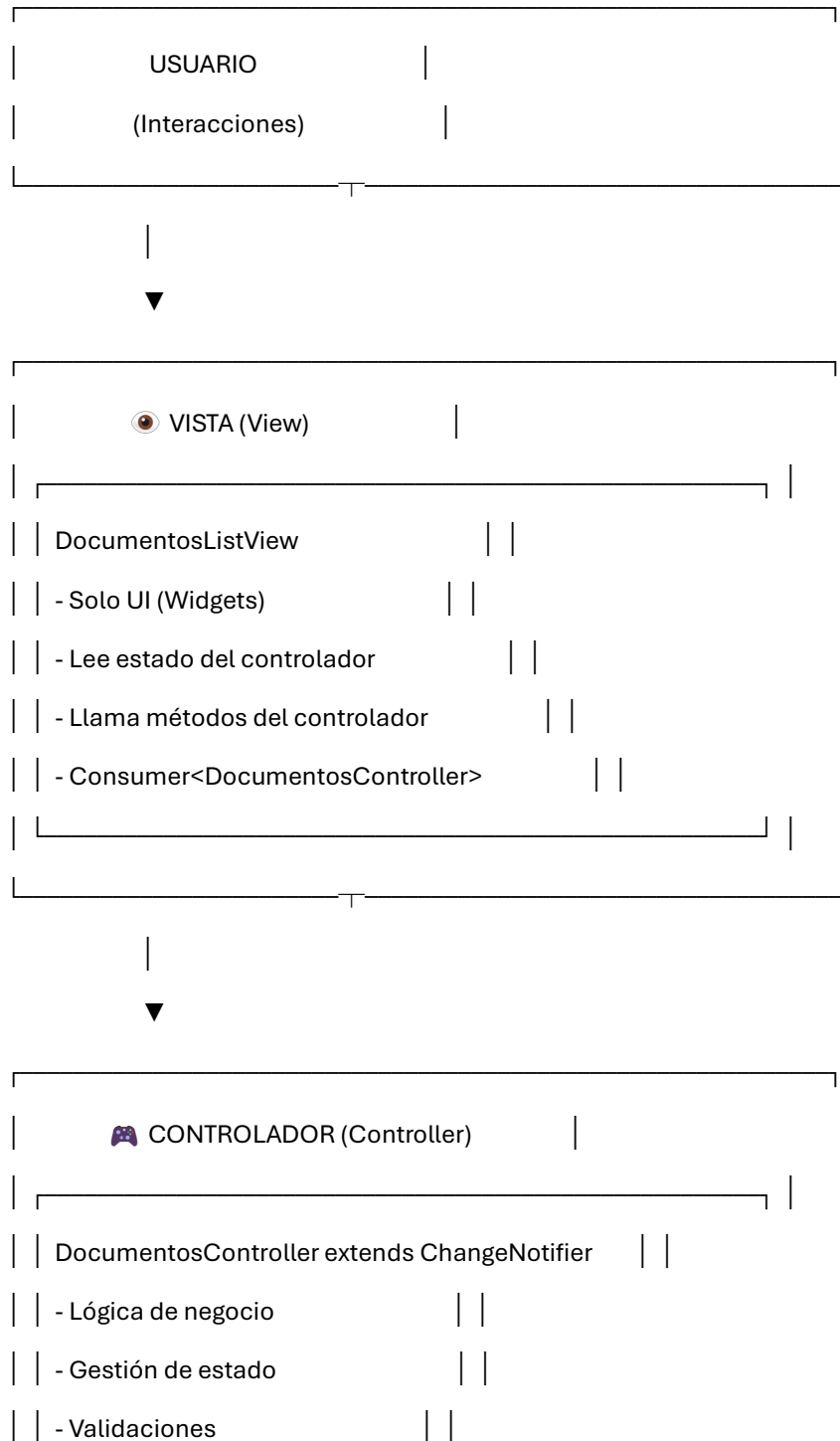
- └─ DocumentosListView  400+ líneas (COMPLETA)
- └─ DocumentoCard  250 líneas
- └─ CarpetaCard  100 líneas
- └─ SubcarpetaCard  70 líneas
- └─ DocumentoFilters  80 líneas

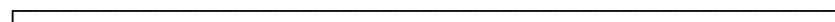
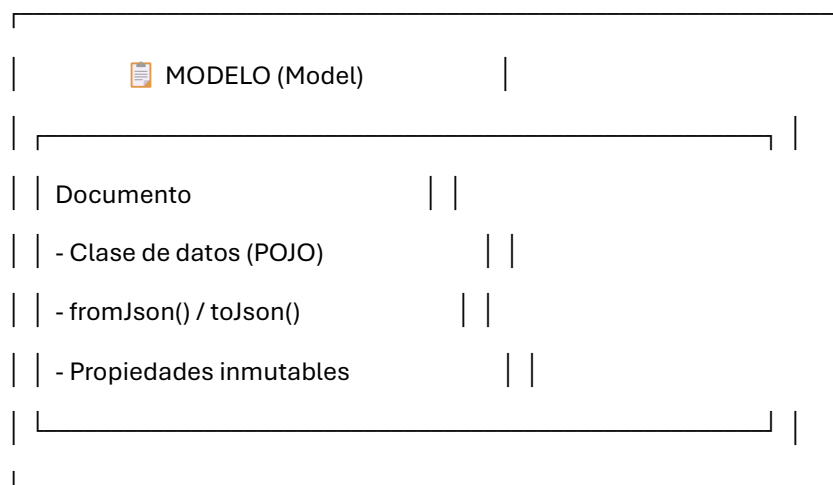
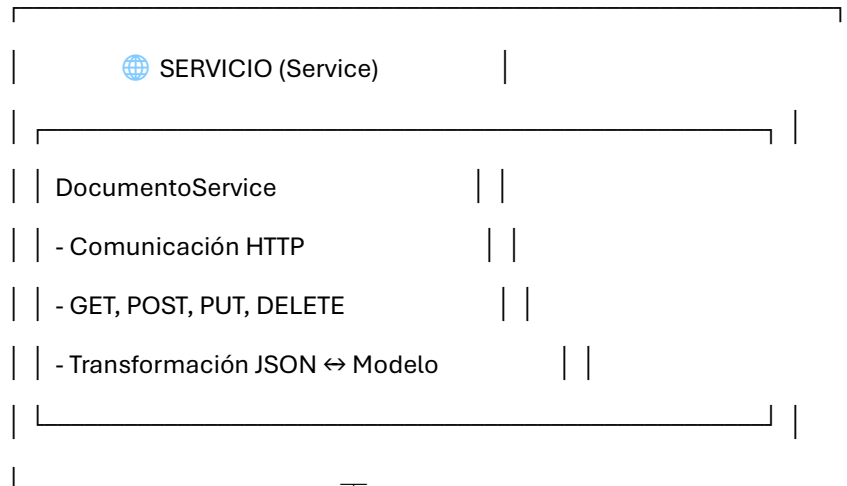
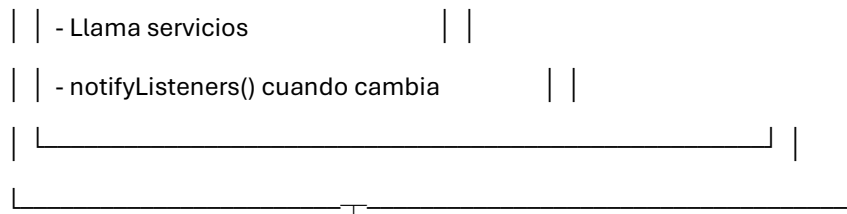
DOCUMENTACIÓN (5 archivos)

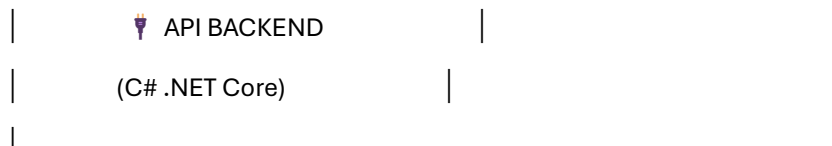
- └─ ARQUITECTURA_MVC.md  200+ líneas
- └─ MVC_IMPLEMENTACION.md  300+ líneas
- └─ .refactor_plan.md  150+ líneas
- └─ controllers/README.md  100+ líneas
- └─ views/README.md  150+ líneas

...

...







Para profundizar exclusivamente en las partes que componen el sistema, podemos desglosar cada capa en sus subcomponentes técnicos y organizativos. El sistema frontend no es solo una división de bloques generales, sino todo un ecosistema de piezas especializadas con roles muy específicos.

1. Capa de Presentación (Interfaz de Usuario)

Esta capa es lo que el usuario ve y toca directamente en la pantalla de su dispositivo. Se divide internamente en dos tipos de partes:

- Pantallas Principales (Screens/Views): Son los contenedores principales de una vista completa. Manejan la estructura base de la pantalla, como la barra superior de navegación, el menú lateral o inferior y el lienzo general en blanco (comúnmente llamado Scaffold). Su deber es organizar la distribución espacial de los elementos y preparar el terreno visual, pero no construirlos uno a uno.
- Componentes Reutilizables (Widgets): Son los ladrillos de construcción. En lugar de escribir el código de un botón repetidas veces en cada pantalla, el sistema tiene archivos que contienen pequeños bloques prefabricados (como "BotonAceptar", "TarjetaDePrestamo" o "BarraDeBusqueda"). Esta parte del sistema asegura que la aplicación se vea uniforme en todas partes y permite al equipo cambiar rediseños masivos modificando un solo archivo central.

2. Capa de Control y Memoria (Gestión de Estado)

Es uno de los bloques vitales de procesamiento en tiempo real. Se estructura en:

- Controladores Globales: Son archivos que mantienen la información viva a través de toda la sesión del usuario. Por ejemplo, un módulo de autenticación recuerda quién inició sesión, tu perfil y tus permisos, evitando consultar la base o obligarte a iniciar sesión cada vez que cambias de ventana.

- Controladores Locales de Tarea: Manejan procesos aislados, como una barra de carga que avanza de 0 a 100% mientras subes un documento. Existen únicamente en la memoria física mientras el usuario ejecuta la tarea; una vez finalizada, esta parte del sistema se autodestruye para liberar la saturación de memoria RAM en el celular o navegador.

- Gestor de Distribución (Provider Pattern): Su función es ser un sistema de tuberías invisible. Se encarga de inyectar sin escalas los datos de los Controladores directamente a los Componentes visuales que están ubicados en partes profundas de una pantalla, evitando que los desarrolladores tengan que pasar coordenadas manualmente ventana por ventana.

3. Capa de Acceso a Datos Ocultos (Servicios y Utilidades)

Aquí es donde el sistema interactúa con factores que el usuario jamás debe ver, como conexiones ocultas:

- Clientes Activos de Conexión (Servicios API): Son clases mecánicas preparadas para cruzar la red de internet. Estas partes tienen el trabajo de interceptar una petición, armar direcciones complejas (URLs), obligar a la conexión a utilizar llaves y tokens de seguridad asimétricos, y manejar elegantemente las crisis: si se cae el internet a la mitad de una orden, es esta capa la que arroja la instrucción de abortar para no congelar la pantalla.

- Utilidades Misceláneas (Utils y Helpers): Son el cajón de herramientas globales. Son funciones cortas que cualquier otra parte del sistema puede pedir prestadas. Contienen piezas como calculadoras de formato de dinero (agregando puntos decimales), traductores que cambian un texto técnico rígido "2026-03-22T10:43" a formatos legibles e incluso rutinas técnicas abstractas.

4. Capa de Estructura Física (Modelos Entidades)

Son los esqueletos matemáticos que el lenguaje de programación protege firmemente.

- Entidades de Dominio Central: Son un reflejo de los conceptos de la empresa. Garantizan mediante tipado fuerte que, por ejemplo, una vez que la pantalla carga a un "Usuario", este objeto tiene un nombre, una edad numérica obligatoria y un folio. El sistema nunca permite procesar a alguien que no cumpla esa "forma".

Dentro del proyecto, la

Capa de Presentación está formada por tres carpetas principales ubicadas dentro de la ruta frontend/lib/. Estos son los archivos reales y concretos que se encargan de dibujar la interfaz de usuario:

1. Carpeta de Pantallas Principales (lib/screens/) Aquí residen las pantallas completas tradicionales de navegación. Algunos de los archivos clave presentes en tu código son:
 - home_screen.dart (El panel principal al que llegas tras iniciar sesión).
 - mis_prestamos_screen.dart (Ubicada en /movimientos, es en la que estás trabajando en este momento).
 - login_screen.dart y register_screen.dart (Las pantallas de entrada y registro).
 - splash_screen.dart (La pantalla inicial animada al abrir la aplicación).
 - notifications_screen.dart y profile_screen.dart. Además, agrupa pantallas en módulos funcionales por carpetas como: /admin, /documentos, /movimientos, /qr y /reportes.
2. Carpeta de Vistas Modulares (lib/views/) Según la implementación reciente de la arquitectura MVC descrita en tu código, esta carpeta contiene las interfaces modernizadas estrictamente separadas de la lógica. Aquí tienes:
 - Subcarpeta /documentos (Que incluye la gran vista modernizada DocumentosListView y sus tarjetas DocumentoCard).
 - Subcarpeta /carpetas (Vistas modulares de subcarpetas y jerarquías).
 - El archivo views.dart (Que funciona como un índice para poder importar rápidamente todos estos diseños).
3. Carpeta de Componentes Reutilizables (lib/widgets/) Aquí están los fragmentos pequeños de interfaz que no son pantallas enteras, sino elementos decorativos y funcionales que usas en todo el sistema para mantener un diseño consistente:
 - sidebar.dart (El menú lateral de navegación general).
 - app_alert.dart (El diseño personalizado de las alertas rojas o verdes que salen en pantalla).
 - glass_container.dart (Cajas visuales con diseño elegante tipo translúcido/vidrio).
 - empty_state.dart (El dibujo que aparece cuando un usuario no tiene documentos ni préstamos).

- loading_shimmer.dart (La sombra animada de carga que se ve mientras llega la información del API).
- animated_card.dart y animated_background.dart (Piezas gráficas de la interfaz).
- Subcarpeta /charts (Los dibujos de gráficas y barras que seguramente usas en la sección de reportes).

Todos los archivos con extensión ".dart" que existan dentro de esas tres carpetas, son los constructores oficiales de tu Capa de Presentación. Todo lo que dibujes o recolques lo harás manipulando exclusivamente estos archivos.

1. Controladores de Módulo (lib/controllers/)

Estos archivos contienen la lógica de negocio específica para cada funcionalidad. Se encargan de procesar las acciones del usuario y manejar los datos temporales del módulo.

- **Módulo de Documentos:**

-

documentos/documentos_controller.dart: Gestiona la lista general de documentos, filtros y búsquedas.

-

documentos/documento_detail_controller.dart: Maneja la lógica específica cuando el usuario abre un solo documento en detalle.

- **Módulo de Carpetas:**

-

carpetas/carpetas_controller.dart: Controla la navegación entre la jerarquía de carpetas y subcarpetas.

- **Módulo Administrativo:**

- Ubicados en la carpeta admin/, estos controladores gestionan usuarios, permisos y configuraciones del sistema.

- **Índice de Controladores:**

-

controllers.dart: Es un archivo que agrupa y exporta todos los controladores para que el resto de la app pueda usarlos fácilmente.

2. Proveedores de Estado Global (lib/providers/)

Estos archivos funcionan como la "memoria central" de la aplicación. Guardan información que debe estar disponible sin importar en qué pantalla se encuentre el usuario.

- **Gestión de Sesión:**

-

auth_provider.dart: Es el archivo más crítico de esta capa. Recuerda quién es el usuario conectado, guarda su token de seguridad y verifica si tiene permiso para entrar a ciertas secciones.

- **Gestión de Preferencias:**

-

theme_provider.dart: Guarda la configuración visual, como si el usuario prefiere el modo oscuro o el modo claro.

- **Almacenamiento de Datos Transversales:**

-

data_provider.dart: Maneja datos compartidos que no pertenecen a un solo módulo, sino que sirven de apoyo para toda la aplicación.

3. El Sistema de Notificación (ChangeNotifier)

Aunque no es un archivo único, casi todos los archivos mencionados arriba heredan o utilizan la clase ChangeNotifier. Esta es la pieza técnica que permite que la "Memoria" se comunique con la "Presentación": cuando un dato cambia en un controlador, se ejecuta el aviso interno que obliga a las pantallas a redibujarse con la nueva información.

Esta estructura asegura que la lógica del sistema (como calcular un préstamo o validar un archivo) nunca se mezcle con el diseño visual, permitiendo que la memoria de la app sea limpia y eficiente.

Los controladores locales en tu sistema son piezas que viven y mueren junto con una pantalla o una tarea específica. A diferencia de los proveedores globales (que siempre están encendidos), estos se destruyen cuando sales de la ventana para ahorrar memoria.

Aquí te detallo cuáles son y dónde están:

1. Controladores de Interfaz Específica (Contextuales)

Se encuentran incrustados en las pantallas o módulos particulares. Su función es manejar el estado de esa vista sin afectar el resto del sistema.

- **Escáner QR (lib/screens/qr/):**

mobile_scanner_widget.dart: Utiliza un controlador específico de cámara (MobileScannerController) que solo se activa cuando se va a leer un código QR.

- **Formularios de Datos (lib/screens/documentos/):**

documento_form_screen.dart: Contiene lógica local para manejar el progreso de carga (subida de archivos) y validación de campos antes de enviarlos al servidor.

subcarpeta_form_screen.dart: Gestiona la creación de jerarquías de forma local.

2. Controladores de Entrada y Animación (Contenidos en el código)

No son archivos separados, sino objetos que viven dentro de los archivos

.dart de las pantallas:

- **Controladores de Texto (TextEditingController):**

- Están presentes en

login_screen.dart,

register_screen.dart y todos los formularios de búsqueda. Se encargan de recordar qué escribió el usuario en cada casilla mientras la pantalla está abierta.

- **Controladores de Animación (AnimationController):**

- Se encuentran en archivos como

splash_screen.dart o animated_background.dart. Controlan el tiempo y la fluidez de los efectos visuales (como el logo desvaneciéndose o el fondo moviéndose).

- **Controladores de Desplazamiento (ScrollController):**

- Se usan en `documentos_list_view.dart` para detectar cuándo el usuario llega al final de la lista y así cargar más documentos automáticamente (infinite scroll).

3. Controladores de Diálogo (`lib/screens/admin/user_dialogs.dart`)

Estos controladores gestionan ventanas emergentes (pop-ups). Por ejemplo, cuando vas a eliminar un usuario o cambiar un permiso, se activa un controlador local que maneja esa confirmación y luego se borra de la memoria al cerrar el diálogo.

COMO SE COMUNICAN

La comunicación entre los archivos de tu sistema sigue un flujo circular y jerárquico muy organizado, basado en un patrón llamado **Inyección de Dependencias** y **Notificación de Estado**.

Aquí te explico cómo se "hablan" los archivos paso a paso:

1. El Registro Central (

main.dart)

Todo nace en

lib/main.dart. Este archivo es el "organizador" principal.

- **Qué hace:** Crea las instancias de todos los **Servicios** (mensajeros) y **Providers** (memoria global).
- **Cómo se comunica:** Envuelve a toda la aplicación en un widget llamado MultiProvider. Esto hace que, por ejemplo, el DocumentoService esté "flotando" en el ambiente, listo para que cualquier otro archivo lo atrape cuando lo necesite.

2. De la Vista al Controlador (El pedido)

Cuando abres una pantalla, como

DocumentosListView:

- **El enlace:** La Vista usa un ChangeNotifierProvider. En su código, busca en el ambiente los servicios que necesita (usando context.read<DocumentoService>()) y se los entrega al **Controlador** en su "mochila" (el constructor).
- **La acción:** Cuando haces clic en un botón de la pantalla (Vista), esta simplemente llama a una función del Controlador: controller.cargarDocumentos(). La Vista no sabe cómo cargar nada, solo sabe a quién pedirselo.

3. Del Controlador al Servicio (El viaje)

El **Controlador** recibe el pedido de la Vista:

- **La orden:** El Controlador llama al **Servicio** que tiene guardado: await _documentoService.getAll().
- **El proceso:** El Servicio viaja por internet al servidor de C#, trae los datos y los convierte usando los **Modelos** (objetos

.dart). Luego se los devuelve al Controlador.

4. El Controlador avisa a la Vista (La respuesta)

Esta es la parte más importante para que la pantalla se actualice:

- **El cambio:** El Controlador guarda los datos nuevos en su memoria interna.
- **El grito:** El Controlador ejecuta una función especial llamada `notifyListeners()`. Es como si gritara a través de un megáfono: *"¡Cambió la información!"*.
- **La reacción:** En la Vista, hay un widget llamado Consumer. Este widget está "escuchando" ese grito. En cuanto lo oye, se activa automáticamente y redibuja la pantalla con los nuevos datos (como la lista de documentos).

Resumen del flujo de comunicación:

1. **Vista** 🖱️ llama al **Controlador** (Acción del usuario).
2. **Controlador** 🖱️ llama al **Servicio** (Lógica y pedido).
3. **Servicio** 🖱️ consulta al **Backend** (Datos reales).
4. **Backend** 📧 responde al **Servicio** (JSON).
5. **Servicio** 🏗️ usa el **Modelo** para armar el objeto y lo entrega al **Controlador**.
6. **Controlador** 🔔 usa `notifyListeners()` para avisar cambios.
7. **Vista** 🔄 se actualiza automáticamente mediante el **Consumer**.

Este sistema permite que, si cambias algo en el Backend, solo tengas que tocar el Servicio o el Modelo, y la Vista seguirá funcionando perfectamente sin saber que algo cambió "detrás de cámaras".

QUE FUNCION CUMPLE CADA PARTE

Para que el sistema funcione de forma impecable, cada una de sus partes tiene una responsabilidad única y sagrada. Aquí tienes el desglose de qué función cumple cada pieza:

1. Vistas (Screens y Widgets)

Responsabilidad: La Presentación.

- **Su única función:** Dibujar y mostrar.
- **Qué hace:** Recibe los datos del controlador y los pone en orden sobre la pantalla usando colores, botones y listas. Captura los toques y textos del usuario para enviarlos al controlador, pero la vista **no sabe qué hacer con ellos**, solo los "pasa".

2. Controladores (Controllers)

Responsabilidad: La Lógica y Memoria Temporal.

- **Su única función:** Decidir y gestionar el estado.
- **Qué hace:** Es el centro de mando. Cuando la vista le avisa que el usuario hizo algo, el controlador decide qué servicio llamar, valida si los datos están bien (ej. si una fecha es correcta) y guarda los resultados en su memoria. Cuando los datos cambian, él es quien "da el grito" (notifyListeners) para que la pantalla se actualice.

3. Servicios (Services)

Responsabilidad: La Comunicación Externa.

- **Su única función:** Ser el mensajero hacia el servidor.
- **Qué hace:** Su trabajo empieza y termina en la conexión a internet. Sabe a qué dirección (URL) debe conectarse del backend, qué llaves de seguridad enviar y cómo manejar errores si el servidor está caído. Una vez que recibe la respuesta, se la entrega al controlador de forma ordenada.

4. Modelos (Models)

Responsabilidad: La Estructura de los Datos.

- **Su única función:** Definir la forma de "los objetos".
- **Qué hace:** Es un molde. Define que un "Prestamista" tiene un nombre (texto) y un ID (número). Su función técnica es traducir el lenguaje del servidor (JSON) al lenguaje de Flutter (Dart) mediante funciones como fromJson. Sin ellos, el controlador no sabría qué datos está recibiendo.

5. Proveedores Globales (Providers)

Responsabilidad: La Memoria Central de la App.

- **Su única función:** Recordar datos que sirven para todas las pantallas.
- **Qué hace:** Se encarga de cosas que no mueren cuando cambias de ventana. Por ejemplo, el AuthProvider recuerda quién eres y mantiene tu sesión activa en todo momento. El ThemeProvider recuerda si prefieres fondo blanco o negro en cualquier parte de la app.

6. Backend (API en C# .NET Core)

Responsabilidad: La Persistencia y Seguridad Absoluta.

- **Su única función:** Procesar las reglas pesadas y guardar en la base de datos.
- **Qué hace:** Es el jefe supremo. Recibe las peticiones del servicio, revisa si el usuario tiene permiso real en la base de datos, guarda los cambios de forma permanente y devuelve las respuestas confirmadas. Es quien hace el trabajo "pesado" que un celular no podría hacer solo.

En resumen:

- **Modelo:** Es la "forma".
- **Servicio:** Es el "viaje".
- **Controlador:** Es el "pensamiento".
- **Vista:** Es el "diseño".
- **Provider:** Es el "recuerdo".
- **Backend:** Es la "realidad" almacenada.

COMO SE MANTIENE, ESCALA Y MODIFICA

La forma en que tu sistema está construido (por capas) es lo que permite que sea un software profesional y no solo un montón de archivos mezclados. Aquí te explico cómo se beneficia en estos tres aspectos:

1. Cómo se Mantiene (Arreglar errores rápido)

Gracias a la **Separación de Responsabilidades**, el mantenimiento es directo y sin adivinanzas:

- **Aislamiento del error:** Si un botón no aparece o está mal alineado, sabes que solo debes abrir las **Vistas** (lib/views/ o lib/screens/). No necesitas tocar el código que calcula datos o llama al servidor.
- **Pruebas rápidas:** Como la lógica está en los **Controladores** (lib/controllers/) y no pegada a los botones, puedes probar si un cálculo de préstamos funciona correctamente sin siquiera encender la pantalla del celular.
- **Código Limpio:** Al tener archivos pequeños (un controlador para documentos, uno para usuarios, etc.), es mucho más fácil leer y entender qué hace cada parte que si tuvieras un solo archivo de 3,000 líneas.

2. Cómo Escala (Crecer sin límites)

El sistema está diseñado para crecer de forma **Modular**:

- **Agregar funciones nuevas:** Si mañana quieres agregar un módulo de "Finanzas", no tienes que modificar nada de "Documentos". Simplemente creas una nueva carpeta con su propio Modelo, Servicio, Controlador y Vista. El sistema es como un juego de LEGO: agregas piezas nuevas sin romper las que ya están armadas.
- **Reutilización:** Si creas un widget (como una "Tarjeta de Alerta" en lib/widgets/), ese mismo archivo te sirve para 50 pantallas diferentes. Si necesitas cambiar el color del sistema, lo haces en un solo lugar y todas las pantallas se enteran.
- **Independencia del Backend:** Si tu base de datos crece de 100 a 1,000,000 de registros, el frontend no sufre. Solo el **Servicio** se encarga de manejar la nueva carga, mientras que el resto de la app sigue funcionando igual.

3. Cómo se Modifica (Cambiar el diseño o la lógica)

Modificar el sistema es seguro porque las piezas están **Desacopladas**:

- **Cambio de diseño total:** Puedes borrar todas tus pantallas y rediseñarlas desde cero (hacerlas más modernas, circulares, con gradientes, etc.). Mientras tus nuevas

pantallas llamen a los mismos métodos del **Controlador**, la aplicación seguirá funcionando perfectamente. **La lógica sobrevive al cambio visual.**

- **Cambio de Servidor:** Si el backend cambia de dirección (URL) o decides usar una base de datos diferente, solo tienes que modificar la configuración en el **Servicio** (lib/services/). Los controladores y las pantallas nunca se enterarán del cambio "detrás de cámaras".
- **Actualizaciones Seguras:** Como cada parte está en su propia caja, existe un riesgo mínimo de que al arreglar algo en la "Lista de Documentos" termines rompiendo accidentalmente el "Login". Los archivos están lo suficientemente separados para evitar el efecto dominó de errores.